What this playbook covers

This is a decisions guide for a RAG system that has to survive contact with real users and real documents, not a tutorial on calling an embeddings API.

Every section names the failure it is guarding against, because most RAG problems only show up under load, staleness, or an adversarial query.

Chunking strategy

Chunk size is a retrieval-precision decision before it is a token-cost decision. Chunks too large dilute the embedding with irrelevant context; chunks too small lose the surrounding meaning a reranker needs.

Split on structure first (headings, sections) and only fall back to a fixed token window when the source has none. Keep a stable chunk id so citations survive a re-index.

Embedding model selection

Benchmark candidate embedding models against your own labeled query/document pairs, not a public leaderboard - domain vocabulary shifts rankings more than most teams expect.

Pin the model version. An unannounced embedding model upgrade silently invalidates your entire vector index, since old and new vectors are not comparable.

Retrieval quality metrics

Precision@k and recall@k answer different questions: whether the top results are relevant, versus whether the relevant document was retrieved at all within k. Track both, not just one.

MRR rewards putting the right answer first, which matters more when only the top 1-2 chunks make it into the prompt due to a context budget.

Hybrid search: dense and sparse

Dense (embedding) retrieval finds semantic matches; sparse (BM25/keyword) retrieval finds exact terms - product codes, error strings, acronyms - that embeddings routinely miss.

Combine both with reciprocal rank fusion rather than picking one; the failure modes of dense and sparse retrieval rarely overlap.

Reranking strategies

A cross-encoder reranker scores query and candidate together, catching relevance signal that a bi-encoder's separate embeddings cannot - at the cost of latency, so only rerank the top 20-50 candidates.

Reranking is the highest-leverage fix for a retrieval pipeline that returns roughly the right documents but in the wrong order.

Handling stale and versioned documents

An index without a document version or supersedes pointer will happily retrieve a deprecated policy alongside its replacement with equal confidence - store and filter on effective-date metadata.

Re-embed on content change, not on a fixed schedule; a scheduled-only refresh means a same-day correction is invisible to retrieval until the next run.

Query rewriting and decomposition

A multi-part question retrieves better as several targeted sub-queries than as one long embedding that averages away each part's specificity.

Rewrite ambiguous pronouns and context-dependent phrasing from conversation history into a standalone query before retrieval, not after.

Grounding and citation strategies

Require the model to cite the chunk id it drew each claim from, and verify programmatically that cited ids were actually in the retrieved set - this catches fabricated citations before the user does.

When no retrieved chunk supports an answer, an explicit 'not found in the provided sources' response is worth more than a fluent guess.

Evaluation harnesses and golden datasets

A golden set of real user queries with human-labeled correct chunks is the only reliable regression signal when you change chunking, embeddings, or reranking - synthetic queries drift from real usage fast.

Re-run the full harness on every retrieval-pipeline change, not just on model upgrades; a chunking tweak can regress recall as easily as a model swap.

Observability for retrieval and generation

Trace retrieval and generation as separate spans with the actual retrieved chunk ids attached - when an answer is wrong, this is what tells you whether retrieval or generation failed.

Log retrieval score distributions over time; a slow drift in average top-1 score often precedes a visible quality complaint by weeks.

Cost control: caching and batching

Cache embeddings for unchanged content and cache full answers for repeated queries; in most production RAG systems a small fraction of queries account for a large share of traffic.

Batch embedding calls at ingestion time rather than one document at a time - most embedding APIs price and rate-limit per request, not per token, at the batch boundary.

Failure modes: hallucination under low context

Hallucination rates rise sharply when retrieval returns weak or irrelevant matches and the model is not instructed to say so - a low-confidence retrieval score should suppress generation, not just get passed through.

Set an explicit minimum similarity threshold below which the system declines to answer rather than generating from noise.

Rollout: shadow mode, canary, rollback

Run a new retrieval or prompt configuration in shadow mode against production traffic, scoring it against the golden set before it ever serves a real answer.

Canary a small percentage of live traffic and compare citation-accuracy and escalation rates against the baseline before a full rollout; keep the previous index version warm for an instant rollback.